Generated from `docs/handoff-guides/markdown/design/dynamics.md` by `tools/guide-publisher`.

Dynamics Handoff Guide

Use this guide when you need to describe **runtime timing, concurrency, ordering, and execution hazards**.

This guide is about **how work runs in time**, not what the business process means.

KNOWLEDGE AREA

This guide helps with **timing, sequencing, and coordination**. Technically, it covers **runtime behavior and temporal coordination**.

Guide family map

Where this guide sits in the full Blueprint Handoff Atlas family.

ROOT & CROSS-CUTTING

Handoff Atlas | Blueprint Bundle | Metamodel Vocabulary | Migrations

DESIGN

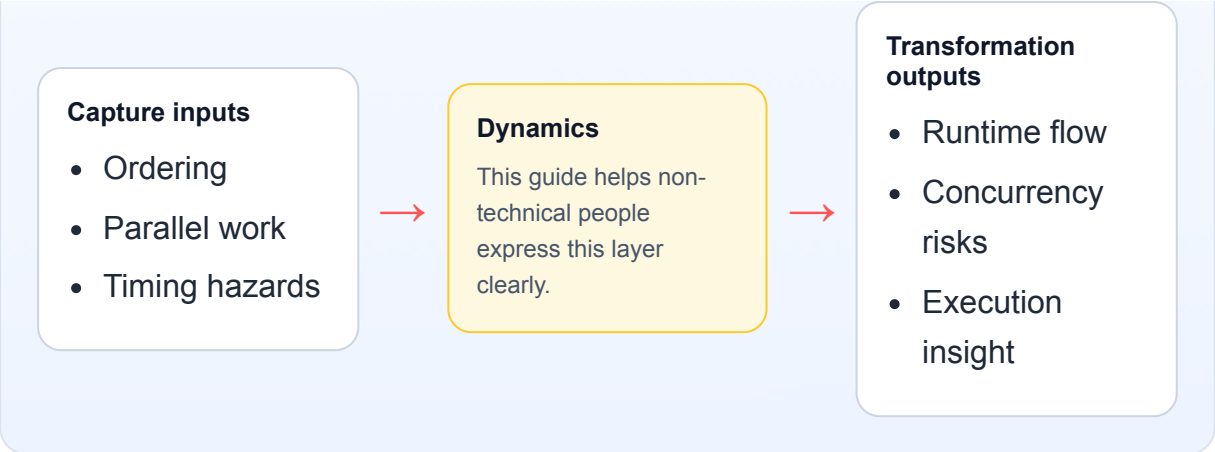
Architecture | Concepts | Domain | **Dynamics** | Infrastructure | Interactions | Models | Quality | Rules | Story

GOVERNANCE

Capability | Decisions | Motivation | Organization | Roadmap | Leverage | Test Cases | Value Stream

Capture → focus → transformation

What to gather first, what this layer focuses on, and what a modeler or AI agent can produce from it.



Main entities in this guide

- **Execution** - the main runtime behavior or execution model being described.
- **Parallelism** - work that can happen at the same time without changing the intended outcome.
- **Ordering** - a sequencing rule that says what must happen before something else.
- **Race Condition** - a timing hazard where different actions can interfere with each other.
- **Resource Profile** - the runtime resource shape, such as CPU, memory, or I/O pressure, that affects timing behavior.
    - Often confused with: story sequencing. **Ordering** here is a runtime constraint, not just a narrative order of steps.

What belongs here

- ordering constraints
- parallel work opportunities
- race conditions or timing hazards
- execution model assumptions
- important resource pressure notes

What does not belong here

- the business narrative of a story
- infrastructure inventory
- static architecture boundaries

Core things to capture

- what must happen before what
- what can happen at the same time
- what must never overlap unsafely
- what execution style is assumed
- where timing or load matters

Core relationships to capture

- operation A -> must happen before operation B
- operations X and Y -> can run in parallel
- concurrent actions -> risk or collision
- runtime behavior -> resource pressure or delay

Modus operandi

Start from real runtime concerns:

1. identify operations that can overlap
2. identify operations that must stay ordered
3. describe known race conditions or timing hazards
4. record why concurrency matters or is dangerous
5. record important resource constraints

Prompt set

- what must happen first?
- what can run in parallel?
- what can go wrong if timing overlaps?
- what delays or bottlenecks matter?
- what execution style best describes the runtime behavior?

Free-text intake template

- **Runtime concern name:**
- **What is happening:**
- **Ordering requirement:**
- **Parallel opportunity:**
- **Risk or hazard:**
- **Expected benefit or impact:**
- **Resource notes:**

Worked example

- Eligibility check and refund estimate can run in parallel after a return request is submitted
- Return approval must happen before the final customer notification is sent
- Support agent review and automated policy evaluation can cause a race condition if request state is stale

Layer-specific handoff guidance

For dynamics, the handoff should make **timing and concurrency** understandable without guesswork.

State explicitly:

- what must happen in order
- what may happen in parallel
- what collisions or timing hazards exist

If the receiver cannot tell why a race condition matters, describe the consequence more clearly.

How this becomes YAML

A technical modeler or AI agent can transform this into ordering, parallelism, race-condition, and execution-model entries tied to the relevant operations.

Common mistakes

- describing business steps instead of runtime timing
- saying "this is parallel" without naming the operations
- omitting the consequence of a race condition
- confusing infrastructure deployment with execution behavior